



Attestium: A Framework for Verifiable Runtime Integrity

An open-source framework for verifiable,
TPM-backed runtime integrity

Forward Email LLC
<https://forwardemail.net>
<https://github.com/attestium/attestium.com>

Foreword

Recent high-profile supply chain attacks, from SolarWinds to the xz-utils backdoor, have exposed a fundamental flaw in modern software security: the gap between build-time security and runtime integrity. We have made significant strides in securing the software supply chain with frameworks like SLSA and tools like Sigstore. These are essential, but they only verify the integrity of software *before* it is deployed.

This whitepaper introduces Attestium, an open-source framework we built to bridge this gap. Attestium produces evidence of what a server-side application is running (its files, its processes, its runtime and its dependencies) that a third party can check against public sources, and binds that evidence to a Trusted Platform Module (TPM) where one is available. The result is continuous, independent verification that the code running in production is the code that was published.

Dedicated to the open-source community and the pursuit of transparency.



Contents

1	Introduction	2
2	The Problem	3
2.1	Build-Time Security is Not Enough	3
2.2	Where Existing Solutions Fall Short	3
3	The Solution	4
3.1	The Attestium Stack	5
3.2	Why GitHub Actions	5
3.3	The Verification Flow	5
4	Systems Architecture	6
4.1	Layer 1: Attestium (Core Library)	6
4.2	Layer 2: Audit Status (Attester and Verifier)	8
4.3	Layer 3: Scheduled Verifier	9
5	Security Architecture	9
5.1	Cryptographic Primitives	9
5.2	What a Result Proves	9
5.3	TPM 2.0 Hardware-Backed Security	10
5.4	Process Memory Integrity	10
5.5	Supply Chain Verification	11
5.6	Threat Model	11
6	Our Security Background	12
6.1	Ansible-Managed Infrastructure	12
6.2	Audits and Continuous Verification	14
6.3	From Hardening to Verification	15
7	The Attestium Stack in Context	16
7.1	How We Compare	16
7.2	Building on Existing Work	16
8	Future Roadmap	17
8.1	Available Now	17
8.2	Planned	17
8.3	Community and Collaboration	18
9	Adoption	18
9.1	Live Transparency at Forward Email	18
9.2	A Call to the Open-Source Community	19
9.3	Preventing the Insider Threat	20
10	Conclusion	20

1 Introduction

We built Attestium because we believe trust in software should be earned, not assumed. It is a **runtime code verification and integrity monitoring library** for Node.js that produces evidence of what a server is running, in a form a third party can check against public sources. Like an element in the periodic table, Attestium is meant to be a basic building block of **attestation**: showing that a system is running exactly the code it claims to run.

The SolarWinds¹ and xz-utils² attacks were not isolated failures; they were symptoms of a systemic problem. Our industry has focused heavily on build-time security—verifying code *before* it’s deployed. This is essential, but it’s only half the story.

What happens after deployment? How can you be certain the code running in your production environment is the same code you so carefully vetted? And even if the files on disk are correct, how do you know the process *actually executing in memory* hasn’t been tampered with? This is the gap where trust breaks down, and it’s the gap we designed Attestium to fill.

Point-in-time audits go stale as soon as the next deploy lands. Continuous deployment calls for continuous verification. The threat is real and growing: Sonatype logged 245,032 malicious open-source packages in 2023 alone³.

Projects like Mullvad’s System Transparency have shown a path forward, proving that verifiable systems are possible⁴. Attestium builds on this foundation, but shifts the focus from the boot process to the runtime environment: the files an application loads, the executable bytes mapped into its processes, and the upstream releases its runtime and dependencies came from.

The urgency of this problem is recognized at the highest levels. Executive Order 14028⁵ and the NIST Secure Software Development Framework⁶ both call for stronger software supply chain security. Attestium is our contribution: a layered, open-source toolkit for verifiable runtime integrity. The following sections describe what each layer checks, what a result does and does not prove, and how we run it against our own production servers.

¹CISA Advisory AA20-352A, “Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations”: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a>

²Andres Freund, “backdoor in upstream xz/liblzma leading to ssh server compromise”: <https://www.openwall.com/lists/oss-security/2024/03/29/4>

³Sonatype, “9th Annual State of the Software Supply Chain”: <https://www.sonatype.com/state-of-the-software-supply-chain/2023/introduction>

⁴Mullvad VPN, “System Transparency is the future”: <https://mullvad.net/en/blog/system-transparency-future>

⁵Executive Order 14028, “Improving the Nation’s Cybersecurity,” Federal Register: <https://www.federalregister.gov/documents/2021/05/17/2021-10460/improving-the-nations-cybersecurity>

⁶NIST SP 800-218, “Secure Software Development Framework (SSDF) Version 1.1”: <https://csrc.nist.gov/pubs/sp/800/218/final>

2 The Problem

The software supply chain is broken. We have focused intensely on securing software before it's deployed, but we've largely ignored what happens after. This is the critical gap where trust evaporates.

2.1 Build-Time Security is Not Enough

We have made great strides with initiatives like SLSA⁷ and Sigstore⁸, creating a chain of trust from source code to binary. But that chain ends at deployment. Google's description of Binary Authorization for Borg makes the same point: alongside checks at deploy time, it continuously re-validates what is already running⁹. A runtime mechanism is not a nice-to-have; it is a necessity.

2.2 Where Existing Solutions Fall Short

Before developing Attestium, we surveyed existing verification, attestation, and integrity monitoring tools. We found nine gaps that none of them closed for our requirements:

1. **Runtime Application Verification Gap:** Most solutions focus on build-time, deployment-time, or infrastructure-level verification. They do not check what an application is running now.
2. **Third-Party Verification Gap:** Few tools produce evidence that an outside party can check independently, without trusting the operator's own dashboard.
3. **Developer Experience Gap:** Hardware-based solutions require specialized knowledge and infrastructure, creating a high barrier to adoption for typical web applications.
4. **Node.js Ecosystem Gap:** Most solutions are language-agnostic or focused on other platforms, with poor integration into Node.js applications and workflows.
5. **Cost and Complexity Gap:** Many solutions are expensive or too complex for small teams.
6. **Granular Monitoring Gap:** File integrity tools monitor files, and application tools monitor performance, but few tie the two to the exact source a deploy came from.
7. **Continuous Verification Gap:** Most solutions provide point-in-time verification, which cannot detect tampering between verification intervals.
8. **Process Memory Integrity Gap:** File checks verify files on disk, not what is running in memory. An attacker who injects code via `ptrace`, writes to `/proc/<pid>/mem`, or loads a

⁷SLSA, "Supply-chain Levels for Software Artifacts": <https://slsa.dev/>

⁸Sigstore, "A new standard for signing, verifying, and protecting software": <https://www.sigstore.dev/>

⁹Google Cloud, "Binary Authorization for Borg": <https://cloud.google.com/docs/security/binary-authorization-for-borg>

library with `LD_PRELOAD` passes every file-level check because the file on disk is unchanged—only the in-memory copy is modified¹⁰.

9. **Supply Chain Provenance Gap:** A package on the npm registry can differ from its public source, and a package on disk can differ from what the registry served. npm provenance attestations help for packages that publish them¹¹, but most packages do not, and nothing in a typical deploy checks the installed files against the exact tarballs the lockfile pinned.

We need a solution that is holistic, layered, and developer-first. This is why we built Attestium.

3 The Solution

We designed Attestium as a layered, practical, open-source toolkit for verifiable runtime integrity. It is not a monolithic system but a set of small modules that can be combined for different environments. It addresses the gaps above with:

- **File Integrity Manifests:** SHA-256 and git object ids for every file in a deployment, so a verifier can compare the tree on a server with a commit in a public repository.
- **Process Memory Integrity:** The executable pages of every file-backed mapping in a process (the binary and every shared library) compared byte-for-byte with the files on disk, plus detection of library and code injection (`LD_PRELOAD`, `LD_AUDIT`, `/etc/ld.so.preload`, `DYLD_INSERT_LIBRARIES`, `AppInit_DLLs`, and `Node.js --require/--import preloads`), attached debuggers, an opened Node.js inspector, and `memfd` payloads.
- **Release Verification:** The Node.js binary compared with the binary inside the official release archive, and every installed npm package compared file-by-file with the registry tarball whose integrity hash the lockfile pins.
- **Hardware Attestation:** TPM 2.0 quotes that bind a report to the platform’s measured state and to a verifier’s nonce, verified in pure Node.js, plus replay of the Linux IMA measurement log against the quoted PCR.
- **Third-Party Verification:** Challenge-response with verifier-chosen nonces, Ed25519-signed responses, and a verifier that fetches its references from public sources instead of trusting the server’s own verdict.
- **Developer-Friendly Design:** A plain npm package with two small dependencies, data-only configuration, and TypeScript definitions.

¹⁰Red Canary, “Playing with Process Memory Integrity on Linux”: <https://redcanary.com/blog/threat-detection/process-memory-integrity-linux/>

¹¹GitHub, “Introducing npm package provenance”: <https://github.blog/security/supply-chain-security/introducing-npm-package-provenance/>

3.1 The Attestium Stack

Our architecture consists of three layers:

1. **Attestium (Core Library)**: A Node.js library that provides the primitives: file manifests, process memory inspection, release and package verification, TPM quotes and verification, IMA log replay, and Ed25519 signing.
2. **Audit Status (Attester and Verifier)**: A single self-contained binary built on Attestium. On a server it collects evidence (`auditstatus ssh`, run through a restricted SSH key); in CI it verifies that evidence against public references (`auditstatus verify`).
3. **Scheduled Verifier (GitHub Actions)**: A workflow in a public repository runs `auditstatus verify` on a schedule, commits the results, and publishes a status badge. We show that badge on our Uptime status page.

3.2 Why GitHub Actions

A key design decision was how to orchestrate these checks without introducing a new processor or subprocessor into our data pipeline. We already trust GitHub — our source code lives there, our CI runs there, and our team authenticates through it every day. Adding another third-party service just to run periodic integrity checks would mean onboarding a new vendor, negotiating a new DPA, and expanding our attack surface for no good reason.

Running the verifier in a public repository’s Actions has another benefit: the workflow definition, its logs, and every result it commits are public. Anyone can read exactly what was checked and when, and anyone can run the same verifier against the same published evidence.

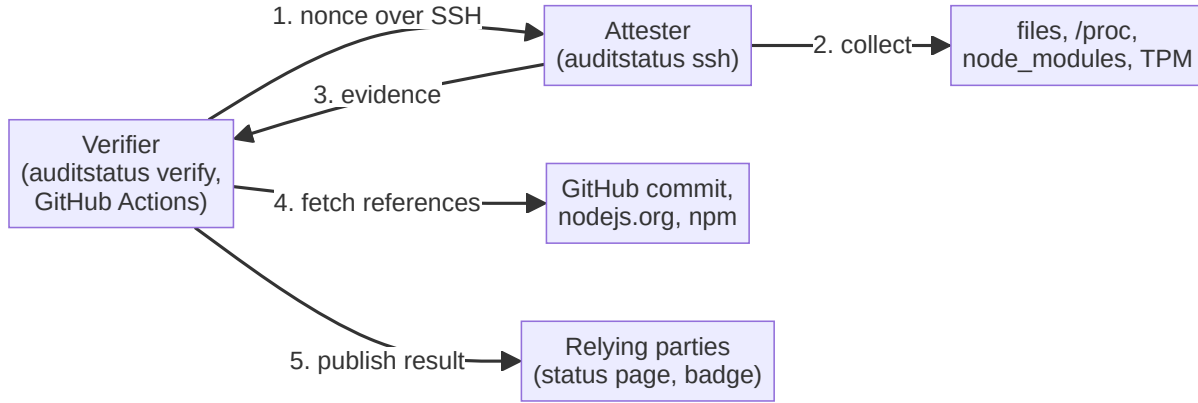
The pattern is not limited to GitHub. `auditstatus verify` is a stateless command that reads a configuration file and writes JSON, Markdown, and badge files. It runs the same way in GitLab CI, Jenkins, or a cron job on a bastion host.

3.3 The Verification Flow

Our verification flow follows the roles of the IETF RATS architecture (RFC 9334)¹². The **Verifier** (`auditstatus verify` in GitHub Actions) generates a fresh nonce and connects to each server over SSH with a key that can only run one command. The **Attester** (`auditstatus ssh` on the server) collects evidence—file hashes, process inspection results, installed package digests, and a TPM quote over the nonce and the evidence digest when a TPM is present—and returns it. The Verifier appraises the evidence against references it fetches itself: the public git commit, the official Node.js

¹²H. Birkholz, et al., “Remote ATtestation procedureS (RATS) Architecture,” IETF RFC 9334, 2023: <https://datatracker.ietf.org/doc/rfc9334/>

release, the lockfile-pinned npm tarballs, and the pinned TPM attestation key. The **Relying Parties** are our users, who read the published results.



The server never decides whether it passed. It only reports what it sees; the verdict comes from a machine and references the operator of the server does not control.

4 Systems Architecture

Here, we detail each layer of the stack, from the core library to the scheduled verifier.

4.1 Layer 1: Attestium (Core Library)

Attestium is a Node.js library with a main module and subpath exports, each covering one verification domain.

4.1.1 Main Module (**attestium**)

- **generateVerificationReport()**: Walks the project tree without following symbolic links and records, for every file, its SHA-256, its git blob id, its mode and its size, plus a digest of the whole manifest.
- **exportVerificationData()** / **compareWithBaseline()**: Exports a baseline (Ed25519-signed when a key is configured) and later reports every added, removed and modified file relative to it.
- **generateVerificationResponse(nonce)**: Answers a verifier's nonce with a signed statement of the current tree digest; **Attestium.verifyVerificationResponse()** checks the signature against a pinned key, the nonce, the age and the digest.
- **generateHardwareAttestation(nonce)**: Produces a TPM quote whose qualifying data is SHA-256(nonce || report digest), binding one specific report to one specific moment on one specific machine.

- **Runtime tracking:** When enabled, records the SHA-256 of each CommonJS module's source exactly as it was handed to the compiler, so later changes to the file on disk can be told apart from what is running.

4.1.2 Process Integrity (**attestium/process-integrity**)

This module inspects running processes rather than the files they were started from.

- **checkExecutablePages(pid):** On Linux, reads every executable, file-backed mapping (the program and each shared library) from `/proc/<pid>/mem` and compares it byte-for-byte with the same range of the file on disk, including the zero fill past the end of the file. A single modified byte of code is reported with its offset. Mappings whose file was deleted or replaced since the process started (usually a package upgrade without a restart) cannot be compared and are reported as such.
- **checkMemoryMaps(pid):** Parses `/proc/<pid>/maps` and reports executable `memfd` mappings, writable and executable file mappings, deleted or replaced backing files, and libraries outside an optional allowlist. Anonymous executable memory is counted but not flagged, because V8's JIT compiler creates it legitimately. On macOS it parses `vmmap`; on Windows it enumerates loaded modules with PowerShell.
- **checkLinkerIntegrity(pid):** Reports `LD_PRELOAD`, `LD_AUDIT` and `/etc/ld.so.preload` on Linux, `DYLD_INSERT_LIBRARIES` on macOS, and `AppInit_DLLs` on Windows. For Node.js it also reports `--require`, `--import` and loader flags and the `--inspect` family, in the process's arguments, `NODE_OPTIONS`, and the `node_args` PM2 started it with. A process that renames itself with `process.title` overwrites its arguments, which can hide the options it started with; this is recognized (a shorter title leaves padding, and any title leaves a single argument) and reported unless the process is a child of the PM2 daemon, whose options are known. Recognizing the daemon by its title is a heuristic that a process of the same user could imitate. Inspector ports named by the options are watched along with the default port, and port 0, which would make an inspector's port unpredictable, is reported.
- **checkTracerPid(pid):** Reports an attached debugger from `TracerPid` on Linux, the traced state flag of `ps` on macOS, and `CheckRemoteDebuggerPresent` on Windows.
- **checkFileDescriptors(pid) / checkListeningSockets(pid):** Report open `memfd` objects (a common fileless payload), deleted files held open, and listening TCP ports, including a Node.js inspector opened at runtime with `SIGUSR1`.

4.1.3 Release Verification (**attestium/release-verification**)

This module compares what is installed with upstream releases.

- **verifyNodeRelease():** Downloads the official release archive listed in `nodejs.org`'s `SHA-SUMS256.txt` (optionally verifying that file's signature with the Node.js release keys), checks

the archive's hash, extracts `bin/node`, and compares it with the binary in use. On Windows, `SHASUMS256.txt` lists `node.exe` directly. A binary rebuilt by a distribution's packager will not match, and is reported as not matching.

- **verifyModules():** Reads the project's `pnpm-lock.yaml` or `package-lock.json`, downloads each registry tarball, checks it against the integrity hash the lockfile pins, and compares its files with the installed package (npm and pnpm layouts). Dependencies bundled inside a package are matched against that package's tarball. Files changed by a pnpm patch and files added by an allowed build script are accounted for explicitly. A dependency pinned to a GitHub commit is compared with the archive of that commit. The two rewrites package managers make are expected: a Windows line ending after a `bin` file's shebang is removed, and Python bytecode caches written by build tools are listed rather than verified. Packages that are not in the lockfile, that come from another non-registry source, or that differ from their reference are reported; a reference that cannot be downloaded leaves the package unchecked, which is not a pass.
- **verifyGlobalPackage(dir):** Verifies globally installed tools such as `npm` and `pm2`. The `npm` and `corepack` copies that ship inside the Node.js archive are compared with that archive; everything else is compared with the tarball named by registry metadata.

4.1.4 TPM and IMA (`attestium/tpm`, `attestium/ima`)

- **Tpm#createAttestationKey():** Creates an Attestation Key under the Endorsement Key and makes it persistent. Its public key is published at enrollment so verifiers can pin it.
- **Tpm#quote():** Produces a signed quote of selected PCRs with the verifier's qualifying data, using `tpm2-tools`.
- **Tpm.verifyQuote():** Parses `TPMS_ATTEST` and checks, in pure Node.js, the signature against the pinned key, the structure's magic and type, the nonce, the digest of the reported PCR values, and any expected PCR values.
- **ima.parseBinaryLog() / ima.backedEntries():** Parse the kernel's IMA measurement log and replay it entry by entry until it reaches the quoted PCR 10 value. The file hashes in that prefix, for PCR 10 and excluding violation entries, are backed by the TPM; entries logged after the quote are not.

4.2 Layer 2: Audit Status (Attester and Verifier)

Audit Status is distributed as a single executable built with Node.js's single executable application support, with published SHA-256 checksums and build provenance for every release. It has two roles:

- **Attester (`auditstatus ssh`):** Installed on each server and invoked only through an SSH key whose `authorized_keys` entry forces this one command. It reads a root-owned configuration

file, accepts nothing from the client except a nonce, and prints evidence as JSON: the project tree manifest, the installed packages, the inspection results for the application's processes (found by owner and working directory), the Node.js binary hash, and a TPM quote when a TPM is present.

- **Verifier (`auditstatus verify`):** Runs in CI. For each configured server it sends a fresh nonce, checks the evidence against public references, and writes a JSON report, a Markdown report, and a Shields.io endpoint badge.

`auditstatus check` runs the same collection locally and prints a readable summary for operators.

4.3 Layer 3: Scheduled Verifier

A GitHub Actions workflow in our public status repository runs `auditstatus verify` on a schedule. It pins the verifier binary by checksum and pins each server's SSH host key, so it cannot be pointed at an impostor. It commits the reports to the repository, which keeps a public history of every result, and opens an issue when an audit fails. The badge at the top of our status page reads the committed badge file.

5 Security Architecture

Our security strategy combines standard cryptographic primitives, a hardware root of trust where one exists, process memory analysis, supply chain verification, and a threat model that states plainly what each layer can and cannot show.

5.1 Cryptographic Primitives

We use standard, well-vetted primitives from Node.js's `crypto` module and the TPM:

- **SHA-256:** For file, package, memory region and manifest digests. Git blob ids (SHA-1) are recorded only to locate files in a git tree; comparisons use SHA-256.
- **SHA-512 / SRI:** For npm tarball integrity, as recorded in lockfiles.
- **Ed25519:** For signed baselines and signed challenge responses, over canonical JSON.
- **TPM 2.0 quotes:** RSA (PKCS#1 v1.5) or ECDSA signatures by a TPM-resident Attestation Key over `TPMS_ATTEST`.

5.2 What a Result Proves

Every check in Attestium runs on the machine it describes. That sets a hard limit on what software alone can prove, so we state the limit directly. Evidence falls into three levels:

1. **Software evidence:** File manifests, process inspection and package verification, collected over SSH and appraised by an independent verifier. This reliably detects drift, failed or

partial deploys, modified files and packages, injected libraries, attached debuggers and open inspectors. It cannot stop an attacker with root access who anticipates the audit: such an attacker can make the collector report whatever they want.

2. **TPM-bound evidence:** The same evidence, bound by a TPM quote to the machine's measured boot state and to the verifier's nonce. This proves the report came from the enrolled machine at that moment and that it booted the expected firmware, bootloader and kernel. It still relies on the collector's honesty for everything after boot.
3. **TPM with IMA:** With the kernel's Integrity Measurement Architecture enabled, the kernel itself hashes files as they are executed or read (according to its policy) and extends each hash into PCR 10 before the file is used. Entries cannot be removed from a PCR, so replaying the log and matching the quoted PCR proves that the logged files are the ones that were loaded, even against a hostile root user. Defeating it requires compromising the kernel or the firmware, which the boot measurements in PCRs 0 to 7 are designed to reveal.

Each result states which level it reached.

5.3 TPM 2.0 Hardware-Backed Security

A Trusted Platform Module is a small, separate processor with keys that cannot be exported and registers (PCRs) that can only be extended, never set. We use it for exactly two things:

- **Attestation keys:** An Attestation Key is created inside the TPM and made persistent. Its public key is recorded when the server is enrolled and pinned in the verifier's configuration. Only that TPM can produce signatures that verify with it.
- **Quotes:** The TPM signs the current values of selected PCRs together with qualifying data we choose. We set the qualifying data to `SHA-256(nonce || evidence digest)`, so a quote cannot be replayed and cannot be moved to a different report.

Pinning the Attestation Key at enrollment is trust on first use. Validating the Endorsement Key certificate chain back to the TPM manufacturer would remove that step; it is on our roadmap.

5.4 Process Memory Integrity

File-level verification answers the question: is the file on disk correct? It cannot answer the more fundamental question: is the code *actually executing in memory* the same code that's on disk?

On Linux, the `/proc` filesystem exposes the internal state of every running process. An attacker who has gained root access—or who has exploited a vulnerability to gain `ptrace` capabilities—can modify a process's executable memory without touching the file on disk. The classic techniques include:

- **ptrace injection:** Attaching to a running process and writing code into its address space.

- **/proc/<pid>/mem writes:** Writing to a process’s memory through the proc filesystem.
- **LD_PRELOAD hijacking:** Forcing the dynamic linker to load a library before all others, overriding any function in any library.
- **memfd_create fileless payloads:** Creating anonymous in-memory files that never touch the disk and executing them.
- **Node.js-specific paths:** Preloading code with `--require` or `NODE_OPTIONS`, or opening the inspector (including at runtime with `SIGUSR1`) and evaluating code through it.

Attestium’s process integrity module detects each of these, and our test suite demonstrates each one against a live process: a byte of `libc` flipped through `/proc/<pid>/mem`, `LD_PRELOAD` and `NODE_OPTIONS` preloads, an inspector opened with `SIGUSR1`, a `ptrace`-attached tracer, and a `memfd` payload.

One limit remains: JIT-compiled JavaScript lives in anonymous memory with no file to compare against. For JavaScript, what matters is the source that was loaded. We cover that by comparing the files on disk with the public commit, by rebuilding generated files (bundled browser code, generated parsers) from that commit and comparing them too, and by reporting files changed after the process started.

5.5 Supply Chain Verification

When you run `npm install express`, you trust that the registry served the package the lockfile recorded, and that nothing modified it on disk afterwards. Attestium checks both. For each installed package, it downloads the registry tarball, requires it to match the lockfile’s integrity hash, and compares the tarball’s files with the installed files one by one.

For every installed package this proves the files are exactly those of the tarball the lockfile pinned, with `pnpm` patches applied exactly; links in `node_modules` must resolve to verified packages. Stray files outside any package are listed but do not fail, since nothing verified loads them. It does not prove that a published tarball was built from the project’s public source; that is the job of `npm` provenance attestations, which we plan to verify where packages publish them.

5.6 Threat Model

We consider four primary adversaries:

1. **External Attacker:** Someone who has gained unauthorized access, for example through a remote code execution vulnerability. Without root, the usual ways to change what runs are reported: modified files, injected libraries, preloads, inspectors and debuggers. Code hidden in the V8 heap, or a process started outside the application’s directory, is outside what the collector sees. With root, software evidence is not enough, and detection depends on IMA and the TPM.

2. **Insider Threat:** A person with legitimate access who modifies files, configuration or running processes. Mistakes and casual tampering are reported by software evidence. A deliberate insider with root who controls the collector can only be caught by TPM-backed IMA measurements, which they cannot erase.
3. **Supply Chain Compromise:** An attacker who alters a package on the registry, in a mirror, or on disk. Lockfile-pinned integrity checks and file-by-file comparison detect any change after the lockfile was written. A malicious version that was published before the lockfile pinned it is outside this check.
4. **Fileless Malware:** Code that lives only in memory. `memfd` payloads, injected libraries and modified code pages are reported. Code hidden inside the V8 JIT heap is not visible to memory comparison.

We do not claim more than this. Each layer narrows what an attacker can do unnoticed. Only the hardware layer holds up against an attacker who already controls the machine.

6 Our Security Background

Before we built Attestium, we spent years hardening our own infrastructure at Forward Email. Security isn't a feature we bolted on — it's the foundation we built everything on top of. Our production environment is managed entirely through open-source Ansible playbooks that anyone can inspect. This section documents what we've done, and why we pair point-in-time audits with continuous automated verification.

6.1 Ansible-Managed Infrastructure

Every server we operate is provisioned and hardened through Ansible. Security measures are codified, version-controlled, and reproducible rather than applied by hand. Here's what our playbooks enforce.

6.1.1 Kernel and System Hardening

We disable core dumps entirely — hard and soft limits set to zero, `fs.suid_dumpable=0`, `Process-SizeMax=0` in `systemd`, and `kernel.core_pattern` piped to `/bin/false`. Transparent Huge Pages are disabled via a `systemd` service. Swap is turned off on most servers; our MongoDB, Valkey and logs servers keep a small emergency swap with `vm.swappiness=1`. Every server gets more than 40 `sysctl` kernel parameters, and database servers get more, including ASLR (`kernel.randomize_va_space=2`), TCP SYN cookies for flood protection, RFC 1337 TIME-WAIT assassination protection, and TCP buffers scaled to available RAM.

`/dev/shm` is mounted as `tmpfs` with `noexec,nosuid,nodev`, so code cannot execute from shared

memory. The root filesystem is mounted with `noatime`, `nodiratime` and `discard`. On database data drives we set the I/O scheduler by device type (`none` for multi-queue devices such as NVMe, `deadline` otherwise) and tune read-ahead. Web, API, CalDAV and CardDAV servers and database servers use TCP BBR congestion control; mail servers use CUBIC with `fq_codel`.

6.1.2 USB Device Whitelisting

The `usb-storage` kernel module is disabled via `modprobe.d` and the `initramfs` is rebuilt to persist this across reboots. We maintain a whitelist of authorized USB devices by `vendor:product` ID in `/etc/security-monitor/authorized-usb-devices.conf`. Any unrecognized USB device triggers an immediate email alert to the team. Udev rules (`99-usb-monitor.rules`) provide real-time detection on top of the 5-minute polling cycle. A datacenter technician plugging in a USB drive gets flagged instantly.

6.1.3 SSH and Access Control

Root login is disabled (`PermitRootLogin no`). Password authentication is disabled — only key-based auth is allowed. The root password is locked. We maintain two users: `devops` (with `sudo`, no password prompt) and `deploy` (with a 4096-bit SSH key, limited privileges). Fail2Ban is configured aggressively: 2 failed attempts trigger a permanent ban (`bantime=-1`) with a 365-day find window, and bans are shared across all of our servers. Alerts go out through a send-only Postfix configuration that listens on no TCP port, accepts mail only from root, and delivers straight to the recipient's mail server with no relay credentials.

6.1.4 Nine Security Monitoring Systems

We run nine independent monitoring systems, all implemented as `systemd` timers with rate-limited email alerts:

1. **System Resource Monitor** — CPU, memory, and disk usage with five threshold levels (75%, 80%, 90%, 95%, 100%), checked every 5 minutes.
2. **SSH Security Monitor** — Failed logins, successful logins, root access, unknown IPs, and after-hours logins, checked every 10 minutes.
3. **USB Device Monitor** — Unknown device detection with `vendor:product` ID whitelisting, checked every 5 minutes plus real-time udev rules.
4. **Root Access Monitor** — Direct root login, `sudo` usage, `su` to root, and privilege escalation attempts, checked every 5 minutes.
5. **Lynis Audit Monitor** — Automated Lynis security audits, run daily.
6. **Package Monitor** — Tracks every package installation and removal, checked hourly.
7. **Open Ports Monitor** — Detects unexpected listening services, checked hourly.
8. **SSL Certificate Monitor** — Checks TLS certificate expiry dates daily.

9. **Outbound Traffic Monitor** — Alerts on sustained outbound traffic above a set limit, checked every minute.

Each monitor uses whitelist files in `/etc/security-monitor/` for authorized IPs, users, USB devices, root users, and sudo users. Rate limiting prevents alert flooding — resource alerts have a 1-hour cooldown per threshold, SSH root access alerts have no cooldown (always alert), and USB alerts have a 1-hour cooldown per device.

6.1.5 Command Logging and Auditing

Every command executed on our servers is logged through multiple layers: `auditd` with custom audit rules, enhanced bash logging in both `/etc/profile.d/` (login shells) and `/etc/bash.bashrc`, `zsh` logging in `/etc/zsh/zshrc.d/`, and `rsyslog` capturing everything to `/var/log/bash-commands.log` with 30-day logrotate retention. If someone runs a command on our servers, we have a record of it.

6.1.6 Automatic Security Updates

Unattended upgrades install security patches automatically. Servers reboot at 08:00 UTC when an update requires it, never while someone is logged in; our MongoDB, Valkey and logs servers do not reboot automatically. We deploy a custom port scan protection script (our own maintained fork). DNS resolves through a local Unbound caching resolver that forwards to Cloudflare and Google over DNS-over-TLS. MongoDB is installed from the official repository and Valkey is compiled from source. Our only external Ansible code is three roles (DNS, fail2ban and sysctl) and three collections, each pinned to an exact version in `requirements.yml`.

6.2 Audits and Continuous Verification

A one-time security audit from a reputable firm costs \$5,000–\$10,000 USD or more. That buys you a snapshot — a report that says “on this date, we checked these things and they looked fine.” The moment the audit ends, the report starts going stale. New code gets deployed, packages get updated, configurations change. The audit doesn’t tell you what happened last Tuesday at 3 AM.

But the cost isn’t the real problem. The real problem is trust.

We manage email. Email is the most sensitive communication channel most people have — password resets, financial statements, legal correspondence, medical records. Giving a third-party auditor SSH access to our production servers means trusting them with access to all of that. Even with the best intentions, an audit can take weeks or months. During that entire window, we’d need to monitor their access, verify they aren’t exfiltrating data, and hope that their own systems haven’t been compromised. We’d essentially need to audit the auditor.

We have since completed a third-party security audit by Cure53 and published the report. Audits and continuous verification answer different questions. An audit is a deep, expert review of the

code and systems at one point in time. Continuous verification checks the running state after every deploy and between deploys. It runs on a schedule, it doesn't get tired, and it needs no interactive access to our servers: the verifier's SSH key can run exactly one read-only command and nothing else.

6.3 From Hardening to Verification

All of the hardening described above — the `sysctl` parameters, the USB whitelisting, the nine monitoring systems, the command logging — these are preventive controls. They make it harder for an attacker to do damage. But they don't answer the fundamental question: is the code running on this server right now the same code that's in our public repository?

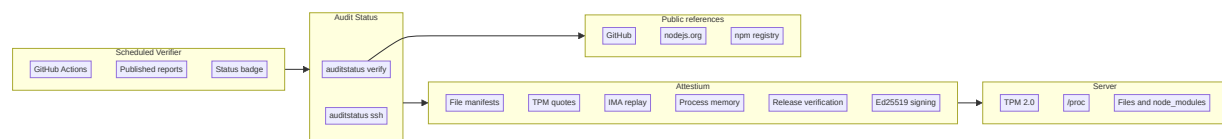
That's the gap Attestium fills. We built it because we needed it ourselves. Every measure in our Ansible playbooks is designed to prevent unauthorized changes. Attestium is designed to detect them, on a schedule, with results that anyone can check against our public repository. Process memory integrity extends that to a deeper question: is the code *actually executing in memory* the same code that's on disk? On a hardened server, the attacker to worry about is not only the one who modifies files, but the one who modifies memory.

7 The Attestium Stack in Context

We didn't build Attestium in a vacuum. It stands on the shoulders of giants in system transparency, runtime attestation, and supply chain security. Here, we discuss how Attestium complements and builds upon this existing work.

7.1 How We Compare

Project	Focus	How We Complement It
System Transparency	Boot-time integrity	We extend the same idea to the running application: its files, its executable memory, and its dependencies.
Keylime	TPM and IMA attestation for cloud infrastructure	We offer a lighter, application-focused toolkit for Node.js, with process memory comparison and package verification alongside TPM quotes and IMA replay.
Sigstore	Signing software artifacts	Sigstore shows what was built and signed; we check what is installed and running after deployment.
SLSA	Supply chain security framework	SLSA hardens the path to a release; we check that the deployed result still matches the release.
Reproducible Builds	Verifiable builds	Reproducible builds let anyone rebuild an artifact; we compare what is deployed with a published source or release.
Socket.dev	Dependency risk analysis	Socket analyzes what packages do; we confirm that the installed files are exactly the lockfile-pinned registry tarballs.



7.2 Building on Existing Work

System Transparency (ST)¹³ pioneered verifiable infrastructure for VPN servers. We take their boot-time integrity model and apply it to the application runtime, extending the chain of trust to the code you actually run and to the code actually mapped into its processes.

Keylime¹⁴ offers powerful TPM-based attestation for cloud infrastructure, including IMA. We provide a more lightweight, application-centric alternative for Node.js developers, and add checks

¹³Mullvad VPN, “Diskless infrastructure in beta (System Transparency: stboot)”: <https://mullvad.net/en/blog/diskless-infrastructure-beta-system-transparency-stboot>

¹⁴Keylime, “A CNCF project for TPM-based remote attestation”: <https://keylime.dev/>

that sit above the kernel’s view: the executable pages of a process compared with the files on disk, Node.js-specific injection paths, and package contents compared with registry tarballs.

Sigstore¹⁵ and **SLSA**¹⁶ are essential for securing the software supply chain at build time. We complete the picture after deployment, by checking that what runs is what was released.

Reproducible Builds¹⁷ ensure that a given set of source code always produces the same binary. As Lamb and Zacchiroli argue, reproducibility is what lets independent parties confirm that a binary came from its claimed source¹⁸. Attestium’s comparisons are the runtime counterpart: they confirm that the binary and packages on a server are the published ones, and that the process in memory has not been modified since it was loaded.

8 Future Roadmap

Here is what Attestium does today and what we plan to add.

8.1 Available Now

- **Process memory integrity:** Byte-for-byte comparison of every file-backed executable mapping with the file on disk, and detection of `LD_PRELOAD/LD_AUDIT/ld.so.preload`, Node.js preloads and inspectors, debuggers, and `memfd` payloads.
- **Release verification:** The Node.js binary compared with the binary inside the official release archive, and installed packages compared file-by-file with lockfile-pinned registry tarballs, with `pnpm` patches, build-script output and bundled dependencies accounted for.
- **Hardware attestation:** TPM 2.0 Attestation Keys and quotes, verified in pure Node.js, and IMA measurement log replay against the quoted PCR.
- **Independent verification:** Audit Status collects evidence through a restricted SSH key and appraises it in CI against public references, publishing every result.
- **Cross-platform inspection:** Full process inspection on Linux; the subset that macOS (`vmmap`, `ps`, `lsof`) and Windows (PowerShell, the registry, `CheckRemoteDebuggerPresent`) expose.

8.2 Planned

- **Endorsement Key certificate validation:** Verify the TPM manufacturer’s certificate chain for the Endorsement Key and use credential activation to prove the Attestation Key lives in the same TPM, replacing trust on first use at enrollment.

¹⁵Sigstore, “A new standard for signing, verifying, and protecting software”: <https://www.sigstore.dev/>

¹⁶SLSA, “Supply-chain Levels for Software Artifacts”: <https://slsa.dev/>

¹⁷Reproducible Builds, “Increasing the integrity of software supply chains”: <https://reproducible-builds.org/>

¹⁸C. Lamb and S. Zacchiroli, “Reproducible Builds: Increasing the Integrity of Software Supply Chains,” IEEE Software, vol. 39, no. 2, pp. 62–70, 2022: <https://doi.org/10.1109/MS.2021.3073045>

- **npm provenance:** Verify Sigstore provenance attestations for packages that publish them, linking a tarball to the source commit and CI run that built it.
- **Built asset verification:** Rebuild front-end assets from the public commit in CI and compare them with the deployed copies.
- **eBPF monitoring:** Observe process and file events as they happen instead of sampling `/proc`.
- **Container-aware verification:** Inspect processes inside containers from the host, across namespaces.
- **Additional runtimes:** Verification strategies for Python, Go and Rust deployments.
- **Confidential computing:** Run the verifier inside AWS Nitro Enclaves or confidential VMs so its own execution can be attested.

8.3 Community and Collaboration

Attestium is an open-source project, and we are committed to building a strong community around it. We welcome contributions and are eager to collaborate with other projects in the software supply chain security space, including Sigstore, SLSA, and Keylime.

9 Adoption

We built Attestium not just for ourselves, but for the entire open-source community. We believe that verifiable runtime integrity is a fundamental building block for a more secure and trustworthy internet. This section outlines our own adoption, our vision for community adoption, and the critical security problems this framework solves for open-source companies.

9.1 Live Transparency at Forward Email

We practice what we preach. At Forward Email, we use Attestium and Audit Status to verify our production servers on a schedule, and we publish every result:

<https://status.forwardemail.net>

A GitHub Actions workflow in our public status repository connects to each server with a key that can run only the read-only evidence collector, and verifies what it receives against our public source code, the official Node.js release, and the npm tarballs our lockfile pins. Where a server has a TPM, the evidence is bound to a TPM quote. The workflow commits a JSON and a Markdown report for every run, so the repository's history is a public record of our production state, and the badge at the top of our status page reflects the latest result.

If an audit fails—a file that differs from the public commit, a modified package, an injected library, altered code in memory, or a failed TPM check—the workflow fails and opens an issue in the status

repository, which notifies our team. Anyone can read the same reports we do.

9.2 A Call to the Open-Source Community

We strongly encourage other open-source companies to adopt this framework. The modern software landscape is built on trust, but that trust is increasingly under attack. High-profile supply chain attacks have demonstrated that build-time security is not enough. We need to be able to verify the integrity of our software *as it runs*—not just the files on disk, but the executable code mapped into our processes, and every dependency against the release it came from.

This is especially critical for companies that are building the next generation of open-source infrastructure and services. We have identified several companies that we believe would be ideal candidates for adopting Attestium, as their business models are built on providing trustworthy, transparent, and secure services. For each, we have included their website, primary GitHub repository, and the main programming languages they use:

- **Supabase** (supabase.com) - github.com/supabase/supabase (TypeScript, Go)
- **Cal.com** (cal.com) - github.com/calcom/cal.com (TypeScript)
- **Documenso** (documenso.com) - github.com/documenso/documenso (TypeScript)
- **Bitwarden** (bitwarden.com) - github.com/bitwarden/clients (TypeScript)
- **Infisical** (infisical.com) - github.com/Infisical/infisical (TypeScript)
- **Jitsi** (jitsi.org) - github.com/jitsi/jitsi-meet (TypeScript, JavaScript)
- **Element** (element.io) - github.com/element-hq/element-web (TypeScript, CSS)
- **Mattermost** (mattermost.com) - github.com/mattermost/mattermost (Go, TypeScript)
- **Ghost** (ghost.org) - github.com/tryghost/ghost (JavaScript, TypeScript)
- **Plausible Analytics** (plausible.io) - github.com/plausible/analytics (Elixir, React)
- **PostHog** (posthog.com) - github.com/PostHog/posthog (Python, TypeScript)
- **Chatwoot** (chatwoot.com) - github.com/chatwoot/chatwoot (Ruby, Vue, JavaScript)
- **Twenty** (twenty.com) - github.com/twentyhq/twenty (TypeScript)
- **Gitea** (about.gitea.com) - github.com/go-gitea/gitea (Go, TypeScript)
- **Rocket.Chat** (rocket.chat) - github.com/RocketChat/Rocket.Chat (TypeScript)
- **Plane** (plane.so) - github.com/makeplane/plane (TypeScript, Python)

By adopting a framework like Attestium, these companies can provide a new level of assurance to their users, customers, and enterprise clients. They can publish continuous, independently checkable evidence that the code on their servers matches their public repositories and the releases they depend on—and, with a TPM and IMA, evidence that holds up even against someone with root access.

9.3 Preventing the Insider Threat

While external threats get the most attention, the insider threat remains one of the most difficult to mitigate. A rogue employee, a compromised third-party vendor, or even a datacenter technician with physical access to a server (an “evil-maid” attack) can bypass traditional security measures and inject malicious code directly into a running application.

This is not a theoretical risk. With SSH access, a malicious actor can easily modify application files, install backdoors, or alter dependencies. More sophisticated attackers can operate entirely in memory—using `LD_PRELOAD` to hijack shared library calls, `ptrace` to inject code into running processes, or `memfd_create` to execute payloads that never touch the filesystem. These techniques leave no trace in traditional file integrity monitoring.

Attestium targets this class of attacks. It compares the deployed files with the public commit. It reads the executable pages of every file-backed mapping from `/proc/<pid>/mem` and compares them byte-for-byte with the files on disk. It reports `LD_PRELOAD` and Node.js preloads, attached debuggers, opened inspectors and `memfd` payloads. And it compares every installed package with the registry tarball the lockfile pinned.

This raises the bar for everyone with access, including us. Casual or careless changes show up in the next public report. A determined insider with root can defeat software-only evidence, which is why the strongest guarantee comes from TPM quotes and IMA: the kernel records what was loaded before the insider can intervene, and the TPM will not sign a history that was edited afterwards.

10 Conclusion

The software supply chain is at a crossroads. We can no longer afford to trust blindly. We must move to a new paradigm of continuous verification and zero-trust.

Attestium is our contribution to this new paradigm. It is a practical, open-source toolkit for runtime integrity that checks a deployment against sources its operator does not control: the public commit, the official Node.js release, and the lockfile-pinned npm tarballs. It compares the executable code mapped into running processes with the files on disk, and it binds its evidence to a TPM where one exists.

We have also tried to be precise about limits. Software evidence detects drift and tampering reliably, but it cannot stand alone against an attacker who controls the machine; TPM quotes and IMA measurements can. Every result says which level of evidence it rests on.

We believe the future of software security is layered and holistic, combining build-time checks, runtime verification, process memory analysis, supply chain verification, and a hardware root of trust. Attestium is a step in this direction, and we are committed to working with the broader



community to build a more secure and trustworthy digital ecosystem for everyone.

The road ahead is long, but we are confident that by working together, we can build a future where software is more secure, transparent, and trustworthy. We invite you to join us.